Agentes para consultas de normativa de pregrado

Entregable 3 · Diseño y evaluación de agentes para consultas normativas

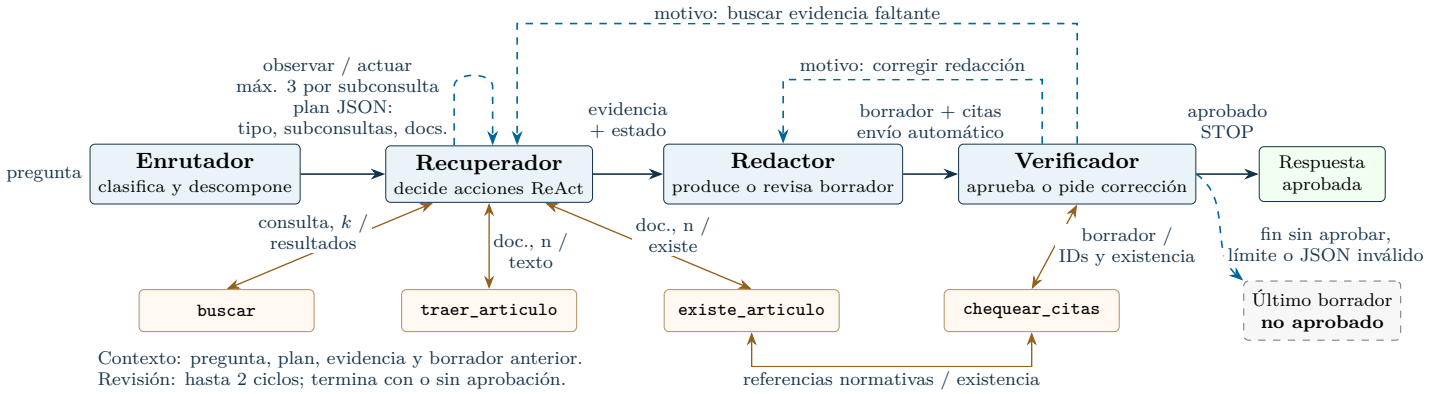
Generative Artificial Intelligence (580694) · Segundo semestre 2026 Álvaro Contreras y Pablo Cortés

Repositorio y reproducción: Deliverable 3 / README **Video (≤3 min):** Pendiente de enlace público

1. Continuidad, tarea y modelo

Continuamos D1/D2: consultas al Reglamento General, al Reglamento Interno de Ingeniería y al Calendario UdeC 2026 (**197 unidades, 201 fragmentos**). Una respuesta correcta es suficiente, está respaldada por citas y no añade afirmaciones falsas; se admite abstención justificada. D2 falla al aplicar excepciones, combinar normas y respaldar respuestas. D3 añade planificación y revisión, manteniendo corpus, embeddings E5 y **Qwen/Qwen3-4B** de pesos abiertos en todos los roles (NF4, T4, temperatura 0,7). El razonamiento nativo está desactivado; CoT se solicita por prompt. Evaluamos las **50 consultas originales y 25 difíciles**.

2. Grafo y condiciones de parada



Continuas: flujo; discontinuas: revisión/parada; bidireccionales: consulta y resultado de herramientas. El Enrutador activa la comprobación de artículos ante premisas falsas. E5 recupera evidencia; no es otro modelo agente.

3. Roles, contratos y técnicas

Cada rol tiene un prompt en `prompts/01-04`. Enrutador, Recuperador y Verificador deciden cómo continuar o detenerse. El controlador envía siempre el borrador al Verificador.

Enrutador. Recibe la pregunta y entrega un plan JSON {tipo, sub_consultas, documentos}. Clasifica, descompone y define el modo de respuesta; no usa herramientas directas. **Few-shot**: guía la descomposición de preguntas que combinan normas.

Recuperador. Recibe subconsulta, evidencia y última observación. Elige una acción JSON {accion, args}: `buscar`, `traer`, `existe` o `suficiente`. Entrega evidencia y estado. Termina cada subconsulta por suficiencia, artículo inexistente o tres acciones. **ReAct**: permite buscar de nuevo cuando se recupera un artículo equivocado.

Redactor. Recibe pregunta, modo, evidencia y objeciones; el prompt no incluye el borrador anterior. Produce Respuesta / Condiciones / Fuente; corrige o declara falta de respaldo. No usa herramientas directas. **CoT**: guía el razonamiento para aplicar excepciones y resolver cálculos; ese bloque se retira de la respuesta final.

Verificador. Recibe pregunta, borrador, evidencia y resultado de `chequear_citas`. Devuelve JSON {aprobado, destino, motivo}: termina o solicita correcciones al Redactor o Recuperador. **Self-refine**: solicita corregir respuestas con citas sin respaldo o información insuficiente.

División del trabajo. Separar planificación, búsqueda, redacción y revisión permite corregir la etapa que falla. Usar el mismo modelo no garantiza independencia entre sus juicios.

4. Ablaciones y alcance

Sin few-shot: retira ejemplos del Enrutador. **Sin CoT:** retira la instrucción del Redactor; conserva la del Verificador. **Sin ReAct:** elimina ciclo y llamadas del Recuperador, manteniendo la búsqueda inicial. **Sin Verificador:** evalúa el primer borrador guardado en las trazas del sistema completo; no es otra ejecución GPU. Estas dos últimas variantes eliminan componentes completos.

Todas usan las mismas consultas y pauta. La semilla depende del prompt: modificarlo cambia también el muestreo. Una ejecución por condición y diferencias de uno o dos aciertos no establecen ventajas robustas. Las consultas se usaron durante el desarrollo; no se demuestra generalización.

5. Resultados

Calificación propuesta con apoyo de Claude y revisada por el equipo, según la pauta de D2 y con justificación por caso. Se evalúa el texto final, independientemente de la aprobación interna.

Configuración	Originales	Difíciles	Llamadas
Directo	1 (2 %)	0 (0 %)	1
RAG D2	40 (80 %)	16 (64 %)	1
Sistema completo	**46 (92 %)**	**20 (80 %)**	**9,5**
Sin few-shot	44 (88 %)	19 (76 %)	10,7
Sin ReAct	45 (90 %)	18 (72 %)	3,9
Sin CoT (Redactor)	46 (92 %)	19 (76 %)	8,3
Sin Verificador	44 (88 %)	19 (76 %)	6,4

Aciertos (porcentaje). Llamadas: promedio al modelo por consulta, sobre 75 entradas; no incluye herramientas automáticas.

Frente a D2: **7 mejoras y 1 regresión** en las originales (+12 puntos porcentuales); **6 mejoras y 2 regresiones** en las difíciles (+16 puntos).

Sin ReAct hay dos aciertos menos en las difíciles; H05 no recupera evidencia para calcular. Sin few-shot hay dos aciertos originales y uno difícil menos. Sin CoT se empata en las originales y el tiempo medio baja de 110 a 55 s. La revisión aporta tres aciertos netos frente al primer borrador, aunque empeora H07.

6. Fallo, costo y límites

H03: plazo para modificar asignaturas en cada semestre. La evidencia seleccionada omite fechas necesarias y el primer borrador inventa 31 y 262 días. El Verificador rechaza, pero las nuevas búsquedas y correcciones no lo resuelven: termina **no aprobado tras 19 llamadas**. Traza: `resultados_v6/trazas_D3.json`, clave `dificil_25:H03`.

Aprobar no equivale a acertar: siete de 64 aprobaciones son incorrectas; nueve aciertos son borradores rechazados. El Verificador recibe hasta 700 caracteres por unidad; comprobar citas no garantiza respaldo semántico. Promedio del sistema: 9,48 llamadas y 110 s por consulta (711 llamadas en 75 consultas, T4).

7. Cómo ejecutar y revisar el sistema

El README de Deliverable 3 explica cómo abrir el notebook en Colab y ejecutar consultas con una GPU T4. Allí se muestran las decisiones de los agentes y la respuesta final. El repositorio incluye código, prompts y resultados para comprobar la evaluación. Los registros guardados permiten revisar qué ocurrió en cada consulta, incluido H03. Como no se fijaron todas las versiones originales de las bibliotecas, una nueva ejecución puede dar respuestas distintas.